

Lập trình Java Hibernate

GV Nguyễn Đức Kiên



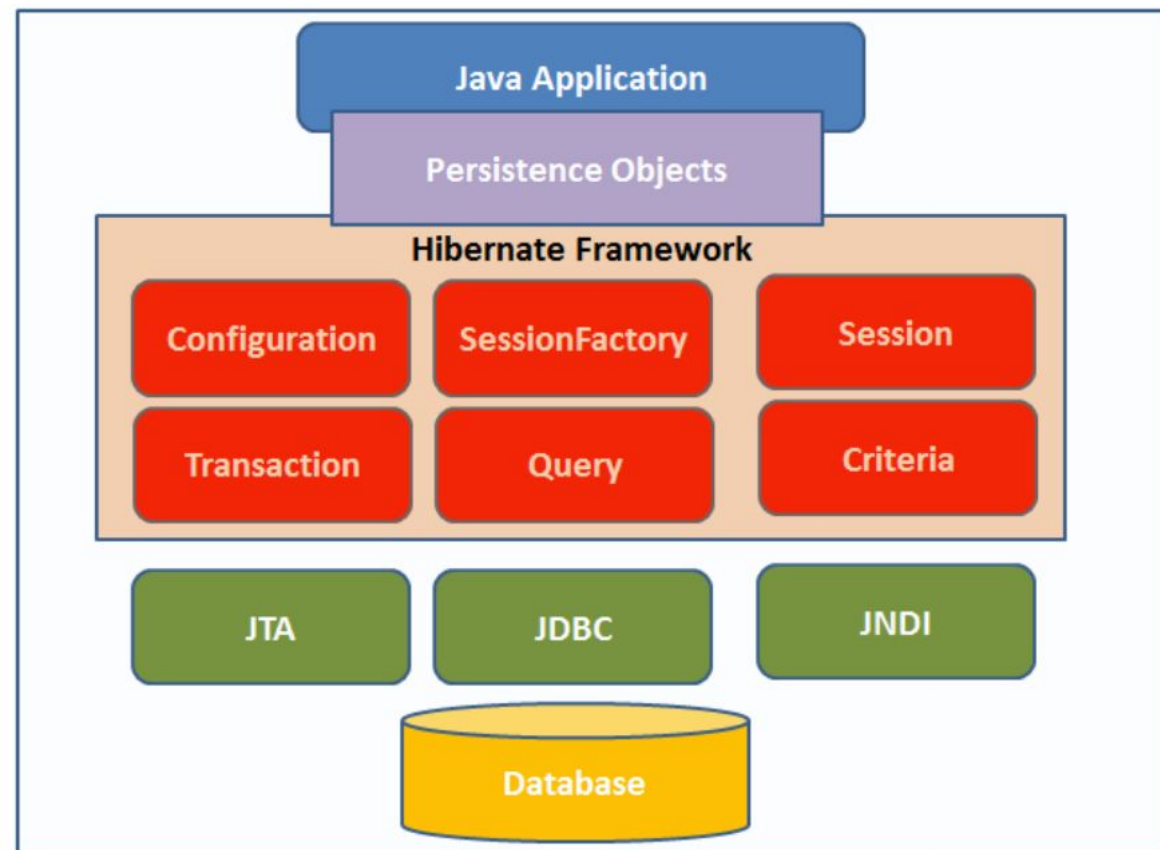
Nội dung bài học

1. Giới thiệu
2. Cách config
3. SessionFactory
4. Session
 - HQL
 - Criteria
5. Transaction

1) Giới thiệu

- **Khái niệm:** Hibernate là một framework ORM (Object-Relational Mapping) cho Java, cho phép chuyển đổi giữa các đối tượng Java và các bản ghi trong cơ sở dữ liệu một cách tự động. Nó giúp giảm thiểu sự phức tạp khi làm việc với cơ sở dữ liệu quan hệ.
- **Ý nghĩa:** Hibernate giúp giảm bớt việc phải viết các mã SQL thủ công, dễ dàng duy trì mã nguồn, và tăng cường khả năng mở rộng của ứng dụng.

Hibernate Architecture



1) Giới thiệu

- **Persistence object**

- ☐ Chính là các POJO object map với các table tương ứng của cơ sở dữ liệu quan hệ. Nó như là những “thùng xe” chứa dữ liệu từ ứng dụng để ghi xuống database, hay chứa dữ liệu tải lên ứng dụng từ database.

- **Hibernate Session**

- ☐ Mỗi một đối tượng session được Session factory tạo ra sẽ tạo một kết nối đến database.

- **Session Factory**

- ☐ Là một interface giúp tạo ra session kết nối đến database bằng cách đọc các cấu hình trong Hibernate configuration. Mỗi một database phải có một session factory.
- ☐ Tỉ dụ nếu ta sử dụng MySQL, và Oracle cho ứng dụng Java của mình thì ta cần có một session factory cho MySQL, và một session factory cho Oracle.

1) Giới thiệu

- Transation
 - ☐ Là transaction đảm bảo tính toàn vẹn của phiên làm việc với cơ sở dữ liệu. Tức là nếu có một lỗi xảy ra trong transaction thì tất cả các tác vụ thực hiện sẽ thất bại.
- Query
 - ☐ Hibernate cung cấp các câu truy vấn HQL (Hibernate Query Language) tới database và map kết quả trả về với đối tượng tương ứng của ứng dụng Java.

2. Cách cấu hình Hibernate

- Cấu hình Hibernate là một phần quan trọng trong quá trình sử dụng Hibernate ORM trong ứng dụng Java.
- Để cấu hình Hibernate, cần thiết lập một số thành phần chính, bao gồm **DataSource**, **EntityManagerFactory**, và **JpaTransactionManager**. Dưới đây là chi tiết về cách cấu hình Hibernate:

2.1) Cấu hình DataSource

- DataSource là thành phần cung cấp kết nối tới cơ sở dữ liệu. Đây là một đối tượng cần thiết để Hibernate có thể kết nối với cơ sở dữ liệu và thực hiện các thao tác CRUD (Create, Read, Update, Delete).
- **Cách cấu hình:**
 1. Sử dụng DriverManagerDataSource (hoặc BasicDataSource từ Apache Commons DBCP) để cung cấp kết nối với cơ sở dữ liệu.
 2. Thông tin kết nối cơ sở dữ liệu sẽ được cấu hình trong file application.properties hoặc application.yml.

2.1) Cấu hình DataSource

- Ví dụ cấu hình DataSource trong ConfigDataSource.java

```
19  @Configuration  * kienn25 *
20  @PropertySource("classpath:application.properties")
21  @ComponentScan(basePackages = "vn.com.t3h")
22  public class ConfigDataSource {
23
24      @Value("${spring.datasource.driver-class-name}")
25      private String driverClassName;
26      @Value("${spring.datasource.url}")
27      private String url;
28      @Value("${spring.datasource.username}")
29      private String username;
30      @Value("${spring.datasource.password}")
31      private String password;
32
33      @Bean  new *
34      public DataSource dataSource() {
35          DriverManagerDataSource dataSource = new DriverManagerDataSource();
36          dataSource.setDriverClassName(driverClassName);
37          dataSource.setUrl(url);
38          dataSource.setUsername(username);
39          dataSource.setPassword(password);
40          return dataSource;
41      }
```


2.1) Cấu hình DataSource

- Thông tin kết nối trong application.properties:

```
application.properties x
1  spring.datasource.url=jdbc:mysql://localhost:3306/shop_book2?useSSL=false&serverTimezone=UTC
2  spring.datasource.username=root
3  spring.datasource.password=root
4  spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
5
6  spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
7  spring.jpa.hibernate.ddl-auto=update
8  spring.jpa.show-sql=true
9  spring.jpa.properties.hibernate.format_sql=true
10
```

2.2) Cấu hình EntityManagerFactory

- **EntityManagerFactory** là thành phần tạo ra EntityManager. Nó quản lý các thực thể (entities) và các phiên làm việc(session) với cơ sở dữ liệu.
- Cần cấu hình EntityManagerFactory để Hibernate biết nơi tìm các thực thể (@Entity), các cấu hình Hibernate và các thuộc tính liên quan đến cơ sở dữ liệu.
- **Cách cấu hình:**
 1. Sử dụng LocalContainerEntityManagerFactoryBean để cấu hình EntityManagerFactory.
 2. Cần chỉ định gói chứa các thực thể cần quét (thông qua setPackagesToScan).
 3. Các thuộc tính Hibernate như hibernate.dialect, hibernate.show_sql sẽ được cấu hình ở đây.

2.2) Cấu hình EntityManagerFactory

- Ví dụ cấu hình EntityManagerFactory trong ConfigDataSource.java:

```
@Bean
public LocalContainerEntityManagerFactoryBean entityManagerFactory(DataSource dataSource) {
    LocalContainerEntityManagerFactoryBean factoryBean = new LocalContainerEntityManagerFactoryBean();
    factoryBean.setDataSource(dataSource);
    factoryBean.setPackagesToScan("vn.com.t3h.entity");
    factoryBean.setJpaProperties(hibernateProperties());
    factoryBean.setJpaVendorAdapter(new HibernateJpaVendorAdapter());
    return factoryBean;
}

private Properties hibernateProperties() {
    Properties properties = new Properties();
    properties.put("hibernate.dialect", databasePlatform);
    properties.put("hibernate.hbm2ddl.auto", ddlAuto);
    properties.put("hibernate.show_sql", showSql);
    properties.put("hibernate.format_sql", formatSql);
    return properties;
}
```


2.3) Cấu hình JpaTransactionManager

- JpaTransactionManager là thành phần giúp Hibernate quản lý giao dịch (transactions). Giao dịch đảm bảo tính toàn vẹn của cơ sở dữ liệu khi thực hiện các thao tác như cập nhật hoặc xóa dữ liệu.
- **Cách cấu hình:**
 1. Cần cung cấp một EntityManagerFactory cho JpaTransactionManager để nó có thể biết được nơi quản lý các đối tượng Entity.
 2. Sau đó, bạn sẽ tạo một JpaTransactionManager và liên kết với EntityManagerFactory.

2.3) Cấu hình JpaTransactionManager

- Ví dụ cấu hình JpaTransactionManager trong ConfigDataSource.java:javaSao chép mã

```
@Bean  @kiennnd25
public JpaTransactionManager transactionManager(EntityManagerFactory entityManagerFactory) {
    JpaTransactionManager transactionManager = new JpaTransactionManager();
    transactionManager.setEntityManagerFactory(entityManagerFactory);
    return transactionManager;
}
```

2.4) Cấu hình Hibernate Properties

- Để Hibernate hoạt động tốt, cần cấu hình một số thuộc tính liên quan đến Hibernate. Các thuộc tính này giúp quản lý các hoạt động của Hibernate như tự động tạo schema, hiển thị câu lệnh SQL và kiểm soát các hành vi khác.
- **Các thuộc tính quan trọng:**
 - ☐ **hibernate.dialect:** Cấu hình dialect của cơ sở dữ liệu, giúp Hibernate hiểu được cách tương tác với cơ sở dữ liệu cụ thể.
 - ☐ **hibernate.hbm2ddl.auto:** Chỉ định hành động Hibernate phải thực hiện khi khởi tạo cơ sở dữ liệu (e.g., update, create, create-drop).
 - ☐ **hibernate.show_sql:** Chỉ định xem có hiển thị câu lệnh SQL được thực thi hay không.
 - ☐ **hibernate.format_sql:** Cấu hình định dạng của câu lệnh SQL.

2.4) Cấu hình Hibernate Properties

- Ví dụ về các thuộc tính này trong hibernateProperties:

```
private Properties hibernateProperties() { 1 usage  👤 kiennd25
    Properties properties = new Properties();
    properties.put("hibernate.dialect", databasePlatform);
    properties.put("hibernate.hbm2ddl.auto", ddlAuto);
    properties.put("hibernate.show_sql", showSql);
    properties.put("hibernate.format_sql", formatSql);
    return properties;
}
```

Tổng kết việc cấu hình Hibernate

- Ứng dụng Java đòi hỏi bạn phải làm việc với một số thành phần quan trọng như DataSource, EntityManagerFactory, và JpaTransactionManager.
- **Các bước cấu hình cơ bản bao gồm:**
 1. Cấu hình DataSource: Cung cấp kết nối cơ sở dữ liệu. Cấu hình
 2. EntityManagerFactory: Thiết lập môi trường Hibernate.
 3. Cấu hình JpaTransactionManager: Quản lý giao dịch.
 4. Cấu hình Hibernate Properties: Tùy chỉnh các hành vi của Hibernate như hiển thị SQL, tự động tạo schema, v.v.

3) SessionFactory

- **Khái niệm về SessionFactory**

- ❑ SessionFactory là một lớp trong Hibernate, chịu trách nhiệm tạo và quản lý các đối tượng Session. Nó giống như một "nhà máy" tạo ra các phiên làm việc với cơ sở dữ liệu.
- ❑ Một SessionFactory thường được tạo ra duy nhất trong vòng đời của ứng dụng. Sau khi được tạo, nó có thể được sử dụng để tạo nhiều Session — mỗi phiên làm việc sẽ thực hiện các thao tác CRUD trên cơ sở dữ liệu.
- ❑ SessionFactory chịu trách nhiệm quản lý các đối tượng Hibernate, cấu hình kết nối với cơ sở dữ liệu, và chịu trách nhiệm xử lý tất cả các liên kết với cơ sở dữ liệu.

Hibernate Architecture:

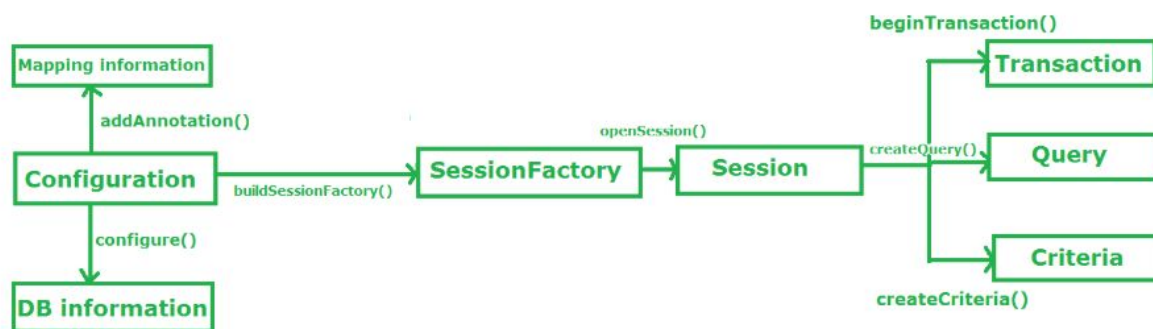


Fig: Hibernate Architecture

3.2) Công dụng và Ý nghĩa

- **Quản lý các Session:** SessionFactory giúp quản lý việc tạo ra các Session, đảm bảo các phiên làm việc với cơ sở dữ liệu có thể được tạo và đóng một cách hiệu quả.
- **Khởi tạo các đối tượng Session:** Mỗi Session được tạo từ SessionFactory sẽ giúp bạn thao tác với cơ sở dữ liệu, thực hiện các truy vấn, lưu trữ và truy xuất các thực thể (entities).
- **Quản lý bộ nhớ và kết nối hiệu quả:** SessionFactory chỉ được tạo một lần trong vòng đời ứng dụng và có thể được sử dụng lại nhiều lần, giúp tiết kiệm bộ nhớ và tối ưu hóa việc sử dụng kết nối cơ sở dữ liệu.

3.3) Cách thức hoạt động của SessionFactory

- **Khởi tạo SessionFactory:** Thông thường, SessionFactory sẽ được khởi tạo trong một lần duy nhất trong ứng dụng, thường là lúc ứng dụng bắt đầu hoặc khi ứng dụng khởi động (ví dụ trong cấu hình Spring hoặc khi ứng dụng web được khởi chạy).
- **Tạo Session:** Sau khi SessionFactory được khởi tạo, bạn có thể sử dụng nó để tạo các Session. Mỗi Session có thể thực hiện các thao tác như lưu trữ, truy vấn, hoặc xóa các thực thể từ cơ sở dữ liệu.
- **Đóng SessionFactory:** Sau khi ứng dụng hoàn thành các tác vụ với cơ sở dữ liệu, bạn có thể đóng SessionFactory để giải phóng tài nguyên. Thông thường, SessionFactory sẽ được đóng khi ứng dụng dừng.

3.4) Các phương thức quan trọng của SessionFactory

- SessionFactory cung cấp một số phương thức quan trọng để thao tác với cơ sở dữ liệu thông qua các đối tượng Session.
- **openSession():**
 - ☐ Phương thức này tạo ra một Session mới và mở kết nối với cơ sở dữ liệu.
 - ☐ Bạn có thể sử dụng Session này để thực hiện các thao tác CRUD trên các đối tượng.
 - ☐ Code ví dụ: `Session session = sessionFactory.openSession();`
- **getCurrentSession():**
 - ☐ Phương thức này tạo và quản lý Session trong phạm vi một giao dịch (transaction).
 - ☐ Khi sử dụng `getCurrentSession()`, Hibernate tự động liên kết Session với giao dịch hiện tại. Điều này làm cho nó trở nên rất hữu ích trong các ứng dụng web hoặc Spring vì nó giúp tự động quản lý giao dịch.
 - ☐ Code ví dụ: `Session session = sessionFactory.getCurrentSession();`

3.4) Các phương thức quan trọng của SessionFactory

close():

- Phương thức này đóng SessionFactory, giải phóng tất cả các tài nguyên đã sử dụng.
- Khi ứng dụng không còn sử dụng Hibernate, bạn cần đóng SessionFactory để giải phóng tài nguyên.
- Code ví dụ: `sessionFactory.close();`

3.5) Lợi ích khi sử dụng SessionFactory

- **Quản lý tài nguyên hiệu quả:** SessionFactory được tạo ra một lần duy nhất và có thể tái sử dụng nhiều lần, giúp giảm thiểu việc tạo và hủy tài nguyên.
- **Tạo Session linh hoạt:** Bạn có thể tạo nhiều Session từ một SessionFactory, giúp dễ dàng thao tác với nhiều phiên làm việc độc lập.
- **Tối ưu hóa hiệu suất:** Hibernate sử dụng một số kỹ thuật tối ưu hóa bộ nhớ và kết nối, ví dụ như caching, giúp giảm tải cho cơ sở dữ liệu và cải thiện hiệu suất.

4) Session

- **Khái niệm về Session:**

- ☐ Session là giao diện chính trong Hibernate để thực hiện các thao tác CRUD với cơ sở dữ liệu.
- ☐ Nó đại diện cho một phiên làm việc với cơ sở dữ liệu trong suốt một giao dịch
- ☐ Mỗi Session được liên kết với một đối tượng Transaction, cho phép bạn thực hiện các thao tác dữ liệu trong một giao dịch cụ thể.
- ☐ Session được sử dụng để truy vấn cơ sở dữ liệu, lưu trữ, cập nhật và xóa các đối tượng trong cơ sở dữ liệu.

4.2) Các phương thức của Session

- Session cung cấp một số phương thức quan trọng giúp bạn thao tác với cơ sở dữ liệu. Dưới đây là các phương thức cơ bản:
- **save():**
 - ☐ Phương thức này lưu một đối tượng vào cơ sở dữ liệu.
 - ☐ Nếu đối tượng chưa có ID (chưa được lưu vào cơ sở dữ liệu), Hibernate sẽ tự động tạo ID cho đối tượng đó.
 - ☐ Code ví dụ: `session.save(productEntity);`
- **update():**
 - ☐ Phương thức này cập nhật một đối tượng đã tồn tại trong cơ sở dữ liệu. Hibernate sẽ kiểm tra sự thay đổi của đối tượng và chỉ thực hiện cập nhật nếu có sự thay đổi.
 - ☐ Code ví dụ: `session.update(productEntity);`

4.2) Các phương thức của Session

- **get():**
 - ☐ Truy vấn cơ sở dữ liệu và trả về đối tượng nếu tồn tại.
 - ☐ Nếu không tìm thấy đối tượng, nó trả về null.
 - ☐ Code ví dụ: `ProductEntity product = session.get(ProductEntity.class, 1);` // Truy vấn theo ID
- **load():**
 - ☐ Tương tự `get()`, nhưng nếu đối tượng không tồn tại, `load()` sẽ ném ra một ngoại lệ `ObjectNotFoundException`.
- **persist():**
 - ☐ Phương thức này giống `save()`, nhưng khác một chút về hành vi: `persist()` sẽ không trả về ID của đối tượng, trong khi `save()` có thể trả về ID.
 - ☐ Code ví dụ: `session.persist(productEntity);`

4.2) Các phương thức của Session

- **createQuery():** Phương thức này dùng để tạo ra các truy vấn HQL (Hibernate Query Language) hoặc JPQL (Java Persistence Query Language).
- **createCriteria():** Phương thức này cung cấp một cách khác để xây dựng các truy vấn thông qua Criteria API (API được Hibernate cung cấp để tạo truy vấn mà không cần viết HQL hoặc SQL).
- Ví dụ:

```
@Override 1 usage  · kienn25
public List<ProductEntity> getAllProducts() {
    // Mở session Hibernate
    try (Session session = sessionFactory.openSession()) {
        // Tạo truy vấn HQL để lấy tất cả sản phẩm
        String hql = "FROM ProductEntity"; // HQL để lấy tất cả bản ghi từ bảng ProductEntity
        Query<ProductEntity> query = session.createQuery(hql, ProductEntity.class);
        // Thực thi truy vấn và lấy danh sách sản phẩm
        List<ProductEntity> productList = query.getResultList();
        return productList; // Trả về danh sách sản phẩm
    }
}
```

4.3) Khái niệm về HQL (Hibernate Query Language)

- HQL (Hibernate Query Language) là một ngôn ngữ truy vấn dành riêng cho Hibernate, được thiết kế để thao tác với các đối tượng (entities) trong Hibernate thay vì các bảng trong cơ sở dữ liệu.
- HQL tương tự như SQL, nhưng thay vì làm việc với bảng và cột trong cơ sở dữ liệu, bạn làm việc với các đối tượng Java và thuộc tính của chúng.
- HQL cho phép bạn xây dựng các câu truy vấn mà không cần phải lo lắng về các chi tiết của cơ sở dữ liệu, như tên bảng hoặc tên cột.
- Điều này làm cho HQL trở nên độc lập với cơ sở dữ liệu, tức là bạn có thể thay đổi cơ sở dữ liệu mà không phải thay đổi mã truy vấn, miễn là cấu trúc entity trong Java không thay đổi.

4.3.1) Các đặc điểm của HQL

- Dựa trên đối tượng (Object-oriented):
 - ☐ Trong HQL, bạn truy vấn đối tượng (entities) thay vì bảng.
 - ☐ Ví dụ: thay vì truy vấn bảng products, bạn truy vấn đối tượng ProductEntity. Các trường của đối tượng (các thuộc tính của class Java) được truy vấn thay vì các cột trong bảng cơ sở dữ liệu.
- Giống SQL nhưng với đối tượng:
 - ☐ Cú pháp của HQL rất giống SQL, nhưng thay vì làm việc với bảng và cột, bạn làm việc với các đối tượng và thuộc tính của chúng.
 - ☐ Bạn có thể sử dụng các phép toán trong HQL như SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, v.v.

4.3.1) Các đặc điểm của HQL

- HQL không phụ thuộc vào cơ sở dữ liệu:
 - ☐ HQL hoạt động với các đối tượng mà Hibernate quản lý và không bị phụ thuộc vào cơ sở dữ liệu cụ thể.
 - ☐ Bạn có thể chuyển đổi giữa các cơ sở dữ liệu mà không phải thay đổi các câu truy vấn.
- HQL hỗ trợ mối quan hệ giữa các đối tượng:
 - ☐ HQL có thể sử dụng các mối quan hệ giữa các entity, như @OneToMany, @ManyToOne, và @ManyToMany.
 - ☐ HQL cho phép bạn dễ dàng tạo các truy vấn với các mối quan hệ này mà không phải sử dụng JOIN phức tạp.

4.3.2) Cú pháp của HQL

- Chọn các đối tượng: HQL sử dụng cú pháp SELECT giống SQL để chọn các đối tượng.

```
@Override 1 usage  kiennnd25
public List<ProductEntity> getAllProducts() {
    // Mở session Hibernate
    try (Session session = sessionFactory.openSession()) {
        // Tạo truy vấn HQL để lấy tất cả sản phẩm
        String hql = "FROM ProductEntity"; // HQL để lấy tất cả bản ghi từ bảng ProductEntity
        Query<ProductEntity> query = session.createQuery(hql, ProductEntity.class);
        // Thực thi truy vấn và lấy danh sách sản phẩm
        List<ProductEntity> productList = query.getResultList();
        return productList; // Trả về danh sách sản phẩm
    }
}
```

4.3.2) Cú pháp của HQL

- Chọn các trường cụ thể: HQL cho phép bạn chọn các trường cụ thể từ đối tượng.

```
String hql2 = "SELECT p.bookTitle, p.author FROM ProductEntity p";  
List<Object[]> results = session.createQuery(hql2).getResultList();
```

- Truy vấn này sẽ trả về danh sách các mảng chứa tên sách và tên tác giả.
- Sắp xếp kết quả với ORDER BY: Bạn có thể sắp xếp kết quả theo một trường cụ thể.

```
String hql4 = "FROM ProductEntity p ORDER BY p.price DESC";  
List<ProductEntity> productList4 = session.createQuery(hql4).getResultList();
```

- Truy vấn này sẽ trả về danh sách sản phẩm được sắp xếp giảm dần theo giá.

4.3.2) Cú pháp của HQL

- Điều kiện tìm kiếm với WHERE: Bạn có thể sử dụng WHERE để thêm các điều kiện vào truy vấn.

```
String hql3= "FROM ProductEntity p WHERE p.price > :price";  
Query<ProductEntity> query3 = session.createQuery(hql3);  
query.setParameter(s: "price", o: 100.0);  
List<ProductEntity> productList2 = query.getResultList();
```

- Truy vấn này sẽ tìm các sản phẩm có giá lớn hơn 100.

4.3.2) Cú pháp của HQL

- Sử dụng JOIN trong HQL: HQL hỗ trợ các phép toán JOIN để làm việc với các mối quan hệ giữa các entity.

```
String hql5 = "SELECT p FROM ProductEntity p JOIN p.category c WHERE c.categoryName = :categoryName";  
Query<ProductEntity> query5 = session.createQuery(hql5);  
query.setParameter(s: "categoryName", o: "Fiction");  
List<ProductEntity> productList5 = query.getResultList();
```

- Truy vấn này sẽ lấy tất cả các sản phẩm thuộc về thể loại "Fiction".

4.3.3) Lợi ích của HQL

Độc lập với cơ sở dữ liệu:

- HQL giúp bạn viết các truy vấn mà không cần lo lắng về chi tiết của cơ sở dữ liệu (như tên bảng và cột). Điều này làm cho ứng dụng dễ dàng chuyển đổi giữa các cơ sở dữ liệu mà không cần phải thay đổi các truy vấn.

Thực thi các truy vấn đối tượng:

- HQL cho phép bạn làm việc trực tiếp với các đối tượng Java, điều này làm giảm sự phụ thuộc vào SQL và tăng khả năng bảo trì mã nguồn.

Hỗ trợ mạnh mẽ mối quan hệ:

- HQL hỗ trợ các mối quan hệ giữa các thực thể như `@ManyToOne`, `@OneToMany`, và `@ManyToMany`. Bạn có thể dễ dàng thực hiện các truy vấn kết hợp các bảng mà không phải viết các JOIN phức tạp trong SQL.

4.3.4) Nhược điểm

- Không hỗ trợ tất cả các tính năng SQL: Mặc dù HQL cung cấp nhiều tính năng mạnh mẽ, nhưng không phải tất cả các tính năng SQL đều có sẵn trong HQL. Ví dụ, một số tính năng SQL nâng cao có thể không được hỗ trợ hoặc bị hạn chế trong HQL.
- Khó debug: Vì HQL được chuyển thành SQL trong Hibernate, việc debug các câu truy vấn HQL có thể khó khăn hơn so với SQL trực tiếp.

4.4) Criteria

Khái niệm về Criteria trong Hibernate

- Criteria là một API trong Hibernate giúp bạn xây dựng các truy vấn linh hoạt và an toàn mà không cần phải viết trực tiếp HQL (Hibernate Query Language) hoặc SQL.
- Thay vì sử dụng chuỗi văn bản để tạo ra câu truy vấn, Criteria cho phép bạn tạo ra truy vấn thông qua các đối tượng Java, cung cấp sự linh hoạt cao và dễ dàng bảo trì mã nguồn.
- API Criteria của Hibernate hỗ trợ việc tạo và thực thi các truy vấn thông qua việc sử dụng các đối tượng CriteriaBuilder, CriteriaQuery, Root, và Predicate, từ đó giúp bạn xây dựng truy vấn một cách linh động và không cần phải lo lắng về các vấn đề cú pháp của SQL hoặc HQL.

4.4) Criteria

- **Ý nghĩa của Criteria**

- ☐ An toàn với kiểu dữ liệu:
- ☐ Bởi vì các truy vấn được xây dựng bằng các đối tượng Java, Criteria đảm bảo rằng các tham số được kiểm tra về kiểu dữ liệu trước khi truy vấn được thực thi.
- ☐ Điều này giúp tránh lỗi cú pháp khi truyền tham số không hợp lệ vào truy vấn. Linh hoạt và dễ bảo trì: Criteria giúp xây dựng truy vấn một cách linh hoạt và dễ dàng bảo trì mà không cần phải viết chuỗi HQL hoặc SQL thủ công.
- ☐ Các điều kiện có thể được thêm, sửa hoặc xóa mà không làm gián đoạn toàn bộ cấu trúc của truy vấn. Dễ dàng tương tác với mối quan hệ giữa các bảng:
- ☐ Criteria cung cấp khả năng xử lý các mối quan hệ giữa các bảng trong cơ sở dữ liệu (chẳng hạn như JOIN, INNER JOIN, LEFT JOIN,...) mà không cần phải viết các truy vấn phức tạp bằng tay

4.4) Criteria

- Các thành phần chính trong Criteria API
- CriteriaBuilder:
 - Đây là đối tượng giúp bạn tạo ra các đối tượng CriteriaQuery, Predicate và các phép toán logic như AND, OR, so sánh, v.v.
 - CriteriaBuilder cũng giúp tạo các điều kiện truy vấn như equal, like, greaterThan, v.v.
 - Ví dụ tạo một CriteriaBuilder từ Session

```
public List<ProductDTO> searchProductsCriteria(Double price, String bookTitle, String publisher, String categoryName) {  
    // Mở session Hibernate  
    try (Session session = sessionFactory.openSession()) {  
        // Tạo đối tượng CriteriaBuilder và CriteriaQuery  
        CriteriaBuilder criteriaBuilder = session.getCriteriaBuilder();  
        CriteriaQuery<ProductDTO> criteriaQuery = criteriaBuilder.createQuery(ProductDTO.class);  
        // Tạo Root cho ProductEntity
```

4.4) Criteria

- **CriteriaQuery:**

- ☐ Đây là đối tượng đại diện cho một truy vấn trong Criteria API.
- ☐ Bạn sử dụng CriteriaQuery để định nghĩa các điều kiện, các truy vấn cần thực hiện (ví dụ: chọn các trường từ một bảng, áp dụng các điều kiện WHERE).

- Ví dụ tạo một CriteriaQuery

```
// Mở session Hibernate
try (Session session = sessionFactory.openSession()) {
    // Tạo đối tượng CriteriaBuilder và CriteriaQuery
    CriteriaBuilder criteriaBuilder = session.getCriteriaBuilder();
    CriteriaQuery<ProductDTO> criteriaQuery = criteriaBuilder.createQuery(ProductDTO.class);
    // Tạo Root cho ProductEntity
    Root<ProductEntity> root = criteriaQuery.from(ProductEntity.class);
    // Tạo các tiêu chí tìm kiếm (Predicates)
    List<Predicate> predicates = new ArrayList<>();
    // Thêm các điều kiện tìm kiếm
    if (price != null) {
        predicates.add(criteriaBuilder.equal(root.get("price"), price)); // Điều kiện price
    }
}
```


4.4) Criteria

Root:

- Root đại diện cho bảng hoặc thực thể trong truy vấn. Đây là nơi bạn xác định các trường của thực thể mà bạn muốn truy vấn.
- Ví dụ tạo Root

```
public List<ProductDTO> searchProductsCriteria(Double price, String bookTitle, String publisher, String categoryName) {  
    // Mở session Hibernate  
    try (Session session = sessionFactory.openSession()) {  
        // Tạo đối tượng CriteriaBuilder và CriteriaQuery  
        CriteriaBuilder criteriaBuilder = session.getCriteriaBuilder();  
        CriteriaQuery<ProductDTO> criteriaQuery = criteriaBuilder.createQuery(ProductDTO.class);  
        // Tạo Root cho ProductEntity  
        Root<ProductEntity> root = criteriaQuery.from(ProductEntity.class);  
        // Tạo các tiêu chí tìm kiếm (Predicates)  
        List<Predicate> predicates = new ArrayList<>();  
        // Thêm các điều kiện tìm kiếm  
        if (price != null) {  
            predicates.add(criteriaBuilder.equal(root.get("price"), price)); // Điều kiện price  
        }  
        if (bookTitle != null && !bookTitle.isEmpty()) {  
            predicates.add(criteriaBuilder.like(root.get("bookTitle"), "%" + bookTitle + "%")); // Điều kiện bookTitle  
        }  
        if (publisher != null && !publisher.isEmpty()) {  
            predicates.add(criteriaBuilder.like(root.get("publisher"), "%" + publisher + "%")); // Điều kiện publisher  
        }  
    }  
}
```


4.4) Criteria

Predicate:

- ☐ Predicate là điều kiện truy vấn trong Criteria API.
- ☐ Bạn có thể sử dụng các đối tượng Predicate để xây dựng các điều kiện như so sánh, tìm kiếm, v.v.
- Ví dụ tạo một Predicate để tìm kiếm sản phẩm có giá lớn hơn một giá trị cụ thể:

```
// Tạo Root cho ProductEntity
Root<ProductEntity> root = criteriaQuery.from(ProductEntity.class);
// Tạo các tiêu chí tìm kiếm (Predicates)
List<Predicate> predicates = new ArrayList<>();
// Thêm các điều kiện tìm kiếm
if (price != null) {
    predicates.add(criteriaBuilder.equal(root.get("price"), price)); // Điều kiện price
}
if (bookTitle != null && !bookTitle.isEmpty()) {
    predicates.add(criteriaBuilder.like(root.get("bookTitle"), s: "%" + bookTitle + "%")); // Điều kiện bookTitle
}
if (publisher != null && !publisher.isEmpty()) {
    predicates.add(criteriaBuilder.like(root.get("publisher"), s: "%" + publisher + "%")); // Điều kiện publisher
}
if (categoryName != null && !categoryName.isEmpty()) {
    // Tạo điều kiện cho categoryName, vì category là một đối tượng con
    Join<ProductEntity, CategoryEntity> categoryJoin = root.join(s: "category", JoinType.INNER);
    predicates.add(criteriaBuilder.like(categoryJoin.get("categoryName"), s: "%" + categoryName + "%")); // Điều kiện categoryName
}
```

4.4) Criteria

Join:

- Đối tượng Join được sử dụng để tạo kết nối giữa các thực thể khi bạn cần làm việc với mối quan hệ giữa các bảng (ví dụ: INNER JOIN hoặc LEFT JOIN).
- Ví dụ kết nối với bảng CategoryEntity:

```
if (categoryName != null && !categoryName.isEmpty()) {  
    // Tạo điều kiện cho categoryName, vì category là một đối tượng con  
    Join<ProductEntity, CategoryEntity> categoryJoin = root.join(s: "category", JoinType.INNER);  
    predicates.add(criteriaBuilder.like(categoryJoin.get("categoryName"), s: "%" + categoryName + "%"));  
}
```

4.4) Criteria

Tổng kết:

- Criteria là một công cụ mạnh mẽ giúp bạn xây dựng các truy vấn linh hoạt và an toàn mà không cần phải viết các chuỗi HQL hoặc SQL.
- Criteria API giúp giảm thiểu các lỗi cú pháp và hỗ trợ dễ dàng trong việc bảo trì và mở rộng mã nguồn.
- Bằng cách sử dụng CriteriaBuilder, CriteriaQuery, và Predicate, bạn có thể xây dựng các truy vấn phức tạp và hỗ trợ các mối quan hệ giữa các bảng trong cơ sở dữ liệu.

So sánh Criteria API và HQL (Hibernate Query Language)

Cả Criteria API và HQL (Hibernate Query Language) đều là những cách thức để thực hiện truy vấn dữ liệu trong Hibernate, nhưng mỗi cái có những điểm mạnh và trường hợp sử dụng riêng biệt. Dưới đây là sự so sánh giữa hai cách này.

Tiêu chí	HQL	Criteria API
Cú pháp	Giống SQL, sử dụng chuỗi truy vấn	Dựa trên đối tượng Java, không cần chuỗi truy vấn
Linh hoạt	Rất linh hoạt, có thể viết truy vấn phức tạp	Linh hoạt, nhưng không trực tiếp viết truy vấn SQL
An toàn kiểu	Cần cẩn thận với tham số, dễ gặp lỗi cú pháp	An toàn kiểu, giảm thiểu lỗi cú pháp
Tương tác mối quan hệ	Sử dụng <code>JOIN</code> để làm việc với mối quan hệ	Hỗ trợ <code>JOIN</code> để làm việc với mối quan hệ
Dễ bảo trì	Có thể khó bảo trì nếu truy vấn phức tạp	Dễ bảo trì hơn, dễ thay đổi cấu trúc truy vấn
Quản lý tham số	Dễ dàng thông qua <code>setParameter</code>	Quản lý tham số dễ dàng thông qua <code>Predicate</code>
Hiệu suất	Tương tự SQL, có thể tối ưu hóa	Tương tự, nhưng dễ dàng kiểm soát tối ưu hơn

5) Transaction

- Trong Hibernate, Transaction là một đối tượng đại diện cho một giao dịch trong cơ sở dữ liệu. Giao dịch là một đơn vị công việc được thực thi với các thao tác như Insert, Update, Delete hoặc Select, đảm bảo tính toàn vẹn của dữ liệu.
- Khi làm việc với cơ sở dữ liệu, một giao dịch có thể bao gồm nhiều thao tác, và các thao tác này phải thành công hoặc bị hủy bỏ (rollback) đồng thời. Nếu một thao tác thất bại, tất cả các thay đổi trong giao dịch sẽ bị hủy, đảm bảo rằng cơ sở dữ liệu vẫn ở trạng thái hợp lệ.
- Hibernate cung cấp Transaction API để quản lý giao dịch, giúp đảm bảo rằng các thao tác đối với cơ sở dữ liệu được thực hiện một cách nhất quán.

5.1) Ý nghĩa của Transaction trong Hibernate

Tính toàn vẹn của dữ liệu:

- ☐ Giao dịch giúp đảm bảo rằng tất cả các thao tác trong một đơn vị công việc được thực thi thành công. Nếu một thao tác trong giao dịch thất bại, tất cả các thao tác trước đó cũng sẽ bị hoàn tác, đảm bảo cơ sở dữ liệu không bị rơi vào trạng thái không hợp lệ.

Tính đồng nhất:

- ☐ Giao dịch đảm bảo rằng mọi thao tác trên cơ sở dữ liệu sẽ được thực hiện một cách đồng nhất, tất cả hoặc không có thao tác nào được thực hiện.

Quản lý giao dịch:

- ☐ Hibernate cung cấp các phương thức để bắt đầu, cam kết và rollback giao dịch, giúp người dùng kiểm soát được các thao tác với cơ sở dữ liệu.

Duy trì trạng thái của đối tượng:

- ☐ Hibernate theo dõi các đối tượng trong suốt vòng đời của một giao dịch. Nếu đối tượng bị thay đổi trong giao dịch, các thay đổi này sẽ được đồng bộ hóa với cơ sở dữ liệu khi giao dịch được cam kết.

5.3) Các thao tác chính với Transaction trong Hibernate

- **Bắt đầu giao dịch:**

- ☐ Giao dịch được bắt đầu bằng cách gọi `session.beginTransaction()`. Sau khi giao dịch bắt đầu, bạn có thể thực hiện các thao tác với cơ sở dữ liệu (chèn, cập nhật, xóa).

- **Cam kết giao dịch (Commit):**

- ☐ Sau khi tất cả các thao tác cần thiết đã hoàn thành và bạn muốn lưu các thay đổi vào cơ sở dữ liệu, bạn sẽ gọi `transaction.commit()`. Điều này sẽ xác nhận giao dịch và lưu các thay đổi vào cơ sở dữ liệu.

- **Rollback giao dịch:**

- ☐ Nếu có lỗi xảy ra trong suốt quá trình giao dịch, bạn có thể gọi `transaction.rollback()` để hủy tất cả các thay đổi đã thực hiện trong giao dịch. Điều này giúp cơ sở dữ liệu quay lại trạng thái ban đầu, không bị ảnh hưởng bởi các thao tác không thành công.

Ví dụ về Transaction trong Hibernate

Giải thích các bước trong ví dụ:

session.beginTransaction(): Bắt đầu một giao dịch. Mọi thao tác với cơ sở dữ liệu sẽ được thực hiện trong phạm vi giao dịch này.

session.save(product): Lưu đối tượng product vào cơ sở dữ liệu. Đây là một thao tác CRUD (Create) trong giao dịch.

transaction.commit(): Cam kết giao dịch, tức là lưu tất cả các thay đổi vào cơ sở dữ liệu.

transaction.rollback(): Nếu có bất kỳ lỗi nào xảy ra trong giao dịch (ví dụ, khi thao tác với cơ sở dữ liệu bị thất bại), gọi rollback() sẽ hủy bỏ tất cả các thay đổi đã thực hiện trong giao dịch.

session.close(): Đóng session sau khi hoàn thành công việc. Việc đóng session rất quan trọng vì mỗi session sử dụng tài nguyên hệ thống và cần phải được giải phóng khi không còn cần thiết.

```
public void saveProduct(ProductEntity product) {  
    // Mở Session  
    Session session = sessionFactory.openSession();  
    Transaction transaction = null;  
    try {  
        // Bắt đầu giao dịch  
        transaction = session.beginTransaction();  
        // Thực hiện thao tác với cơ sở dữ liệu  
        session.save(product);  
        // Cam kết giao dịch  
        transaction.commit();  
    } catch (Exception e) {  
        // Nếu có lỗi, rollback giao dịch  
        if (transaction != null) {  
            transaction.rollback();  
        }  
        e.printStackTrace();  
    } finally {  
        // Đảm bảo đóng session  
        session.close();  
    }  
}
```


5.4) Các trạng thái của giao dịch trong Hibernate

Active: Giao dịch đang được thực hiện (sau khi gọi `beginTransaction()` và trước khi gọi `commit()` hoặc `rollback()`).

Committed: Giao dịch đã được cam kết, tức là tất cả các thay đổi đã được lưu vào cơ sở dữ liệu.

Rolled Back: Giao dịch đã bị hủy, tất cả các thay đổi đã thực hiện đều bị hoàn tác.

5.5) Các lợi ích khi sử dụng Transaction trong Hibernate

Đảm bảo tính toàn vẹn của dữ liệu: Hibernate cung cấp các phương thức để quản lý giao dịch, đảm bảo rằng tất cả các thay đổi trong cơ sở dữ liệu sẽ được thực thi hoặc hủy bỏ đồng thời.

Quản lý giao dịch tự động: Hibernate giúp bạn dễ dàng quản lý giao dịch mà không cần phải tự viết mã SQL để bắt đầu, cam kết, và rollback giao dịch.

Khả năng mở rộng và hiệu suất: Khi làm việc với các giao dịch, Hibernate có thể tối ưu hóa các thao tác cơ sở dữ liệu và giúp bạn xử lý các thao tác phức tạp một cách hiệu quả.

